Davante

Desarrollo de Aplicaciones Web
Desarrollo Web Entornos Cliente

Tema 4.
Operadores del lenguaje y bucles

Diego Rodero Pulido

Tarea 4. Calculadora y bucles

Tema 4. Operadores del lenguaje y bucles · Tarea individual · Calificación: 5 puntos de rúbrica + hasta 5 puntos en la defensa en clase.

Objetivo

Practicar los operadores y los bucles del tema en una página interactiva maquetada con Bootstrap. Al terminar la tarea debes ser capaz de:

- Operar con los operadores aritméticos, incluidos `**` y `%`, y controlar con `if` los casos que no tienen sentido, como dividir entre 0.
- Comparar valores con `>`, `<` y `===`, combinar condiciones con `||` y elegir entre dos opciones con el operador ternario.
- Repetir un bloque con `for`, `while` y `do…while`, y explicar en qué se diferencian.
- Maquetar una página con la rejilla y los componentes de Bootstrap que se vea bien en el móvil.

Qué vas a construir

En la carpeta `tema04` de tu repositorio de DWEC vas a crear una página, «Calculadora y bucles», con dos casillas para escribir números, una zona de resultado y un botón para cada operación y cada bucle. Cada botón llama a una función de `js/app.js`, que muestra el resultado en la página y deja su rastro en la consola.

Para empezar, **te doy una plantilla básica.** La maquetación es cosa tuya.

- Está en la carpeta `tema04_plantilla` del GitHub de la asignatura: https://github.com/DRodero/DWEC_2627. Cópiala en tu repositorio y cámbiale el nombre a `tema04`. Trae esto:

```
tema04_plantilla/
├── index.html     las casillas, la zona de resultado y el botón Sumar
├── js/
│   ├── ayudas.js  las funciones de ayuda: este archivo no se toca
│   └── app.js     sumar() hecha como modelo; las demás, con TODO
├── capturas/      imágenes de relleno con el nombre de cada captura
└── README.md      el guion del README, con sus apartados
```

- Todo lo que va entre **[corchetes]** es un hueco: sustitúyelo por lo tuyo. Y cuando completes un `TODO`, borra el comentario.

Las funciones de ayuda

Leer lo que se escribe en una casilla y escribir en la página es trabajar con el DOM, y eso lo veremos más adelante. Para que ahora te centres en los operadores y los bucles, `js/ayudas.js` trae cinco funciones ya hechas. No hace falta que sepas cómo funcionan por dentro: úsalas como usas `alert()` o `console.log()`.

Función	Qué hace
`leerNumero("numero1")`	Devuelve el número escrito en la casilla con ese `id`. Si la casilla está vacía, devuelve `NaN`.
`mostrar(texto, color)`	Borra la zona de resultado y muestra el texto en un aviso de Bootstrap. El color es opcional: `"success"` (verde, el de por defecto), `"danger"` (rojo), `"info"`, `"warning"`...
`anadirLinea(texto)`	Añade una línea a la zona de resultado, sin borrar lo que ya hay.
`anadirBadge(texto, color)`	Añade un badge de Bootstrap a la zona de resultado, sin borrar lo que ya hay. Color por defecto: `"primary"`.
`limpiarSalida()`	Vacía la zona de resultado.

Para que funcionen, no cambies los `id` de las casillas (`numero1` y `numero2`) ni el de la zona de resultado (`salida`), y deja `ayudas.js` enlazado antes que `app.js`, como viene en la plantilla.

Requisitos de la página. **Bootstrap ya no es opcional:**

- Tu **nombre y apellidos** en la `navbar`, en el `<title>` y en el pie de la página.
- La **rejilla de Bootstrap** (`row` y `col-*`): en el móvil, las cards van una debajo de otra; en pantalla grande, repartidas en columnas. Compruébalo con el modo dispositivo de F12.
- Una `card` para los números, otra para el resultado y una por ejercicio: calculadora, comparaciones y bucles.
- Botones de Bootstrap con un color para cada tipo (operaciones, comparaciones y bucles), y los de los bucles agrupados con `btn-group`.
- Sin CSS propio: todo el aspecto lo pone Bootstrap.
- `let` y `const`, nunca `var`; comparaciones con `===`; comillas rectas; y cada función deja su rastro en la consola con `console.log()`.

Ejercicio 1. Calculadora

1. Añade los botones Restar, Multiplicar, Dividir, Potencia y Módulo, y escribe sus funciones siguiendo el modelo de `sumar()`: leer los dos números, comprobar que están y mostrar la operación completa, por ejemplo «12 + 5 = 17».

2. Potencia, con el operador `**`, y Módulo (el resto de la división), con `%`.
3. Dividir y Módulo controlan el 0: si el número 2 es 0, muestran un error en rojo. En JavaScript, `5 / 0` no da error, da `Infinity`, y `5 % 0` da `NaN`: por eso tienes que controlarlo tú con un `if`.
4. Si falta algún número, todas las operaciones muestran un error en rojo en lugar del resultado.

Ejercicio 2. Comparaciones

1. «¿Cuál es mayor?»: con `if`, `else if` y `else`, di cuál de los dos números es mayor o si son iguales.
2. «¿Par o impar?»: solo con el número 1. Usa `%` para saber si es par y el operador ternario (`condición ? "par" : "impar"`) para elegir la palabra. Si el número tiene decimales, avisa de que no es ni par ni impar.

Ejercicio 3. Bucles con texto

El número 1 dice cuántas veces se repite cada bucle.

1. Tres botones, `for`, `while` y `do…while`, cada uno con su bucle. En cada vuelta, una línea con `anadirLinea()`: «for · repetición 3 de 5».
2. Cuenta las vueltas en una variable `contador` y, al terminar, añade una línea con el total: «Total: 5 repeticiones». Con el ternario, que diga «1 repetición» en singular.
3. Antes de repetir, comprueba el número: si falta, si tiene decimales o si pasa de 20 (la constante `MAXIMO_REPETICIONES` de la plantilla), muestra un error en rojo. Combina las condiciones con `||`.
4. Prueba los tres con 0 y con un número negativo. El `do…while` tiene que repetirse una vez aunque el número sea 0 o negativo: explica en el README por qué.

Ejercicio 4. Bucles con badges

1. Otros tres botones que hacen lo mismo que los del ejercicio 3, pero cada vuelta añade con `anadirBadge()` un badge con el número de la repetición.
2. Los pares, de un color y los impares, de otro. El color se elige con el operador ternario.
3. Al terminar, la misma línea con el total que en el ejercicio 3.

Evidencias y reflexión (README.md)

En el `README.md` de la carpeta explicas lo que has hecho y lo demuestras; la plantilla trae el guion con los apartados. Las capturas van en la carpeta `capturas/`, tienen que estar hechas en tu equipo y cada una lleva debajo una o dos líneas que expliquen qué se ve.

1. Cinco capturas propias, que sustituyen a las imágenes de relleno de `capturas/` con el mismo nombre: la página entera, con tu nombre visible; la página en el móvil, con el modo dispositivo de F12; el error de dividir entre 0, con la consola abierta; los tres bucles de texto con el número 0, con la consola abierta; y un bucle con badges.
2. Una reflexión de 5 a 8 líneas: ¿en qué se diferencian `for`, `while` y `do…while`, y cuándo usarías cada uno? Pon como ejemplo lo que pasó con el 0.
3. Fuentes consultadas, con su enlace, y el apartado «Uso de IA» si corresponde.

Entrega

- Todo va en la carpeta `tema04` de tu repositorio público de DWEC, con esta estructura:

```
tema04/
├── index.html     la página, maquetada con Bootstrap
├── js/
│   ├── ayudas.js  el de la plantilla, sin cambios
│   └── app.js     una función por botón
├── capturas/      las cinco capturas
└── README.md      capturas, reflexión y fuentes
```

- Sube el trabajo en varios commits a medida que avances (tres como mínimo), con mensajes que digan qué cambia: «Calculadora: las seis operaciones», «Bucles con texto»…
- En la tarea del campus virtual entregas solo la URL de la carpeta: `https://github.com/tu-usuario/tu-repositorio/tree/main/tema04`
- Extensión orientativa del README: de 200 a 400 palabras, más las capturas.
- Se corrige el último commit anterior a la fecha límite publicada en la propia tarea del campus; lo que subas después no cuenta. No se admiten entregas por otros medios.

Criterios de evaluación (rúbrica)

La tarea se califica sobre 5 puntos con la rúbrica siguiente. Los otros 5 puntos, hasta el 10, se obtienen en la defensa en clase.

Criterio (puntuación máxima)	Insuficiente	Adecuado	Excelente
1. Maquetación con Bootstrap (1 p)	0 – 0,25. Sin rejilla o no se adapta al móvil, no se ve tu nombre o no se distinguen los ejercicios.	0,5 – 0,75. Rejilla con row y col-*, una card para los números, otra para el resultado y una por ejercicio, y botones con color.	1. Maquetación limpia que se ve bien en el móvil y en pantalla grande, colores con sentido, bucles en btn-group y detalles como iconos.
2. Calculadora y control de errores (1 p)	0 – 0,25. Faltan operaciones o no se controla la división entre 0.	0,5 – 0,75. Las seis operaciones funcionan, con ** y %, y se controla la división entre 0.	1. Además, controla el 0 en el módulo y las casillas vacías en todas, y cada operación deja su rastro en la consola.
3. Comparaciones y operador ternario (0,5 p)	0. No funcionan o falta alguna.	0,25. «¿Cuál es mayor?» con los tres casos y «¿Par o impar?» con %.	0,5. Par o impar con el ternario y aviso cuando el número tiene decimales.
4. Bucles for, while y do...while (1 p)	0 – 0,25. Faltan bucles o se usa el mismo tipo de bucle en todos los botones.	0,5 – 0,75. Los seis botones funcionan, cada uno con su bucle, con contador y total.	1. Comprueba el número antes de repetir, el do...while entra una vez con 0 y los badges alternan color con el ternario.
5. Evidencias, reflexión y GitHub (0,5 p)	0. Sin capturas propias, reflexión genérica o todo el trabajo en un único commit.	0,25. Cinco capturas propias comentadas, reflexión de 5 a 8 líneas y varios commits.	0,5. Capturas con tu nombre visible, reflexión con el ejemplo del 0 y commits que muestran el avance.
6. Calidad del trabajo entregado (1 p)	0 – 0,25. Código desordenado o sin sangrar, comillas curvas, var, restos de la plantilla (huecos o TODO sin quitar) o README descuidado y con faltas.	0,5 – 0,75. Código ordenado y legible, README completo y pocas erratas.	1. Código limpio y comentado, nombres claros en camelCase y README cuidado, sin faltas.

Defensa en clase (hasta +5)

La defensa vale **la mitad de la nota**: es donde demuestras que sabes hacer lo que has entregado. No se trata de repetirlo: te pediré uno o dos cambios pequeños, los harás en el momento explicando lo que haces y te preguntaré por tu código.

- **Cuándo:** en la sesión de clase que indique después del cierre del plazo. Unos cinco minutos por persona.
- **Requisito:** solo puedes defender la tarea si la has entregado en plazo. Si no la defiendes, tu nota es la de la rúbrica (máximo 5).
- **Cómo:** abres tu página desde la versión entregada, la enseñas funcionando con la consola abierta, haces en tu propio código el cambio que te pida y respondes a lo que te pregunte sobre él.
- **Sin IA:** durante la defensa desactiva Copilot o cualquier autocompletado con IA. Puedes consultar los apuntes y la documentación de MDN.

Ejemplos de lo que te puedo pedir

- Añadir una operación nueva, por ejemplo la media de los dos números, con su botón.
- Cambiar un `for` por un `while`, o al revés, sin que cambie el resultado.
- Predecir cuántas veces se repite un bucle con el número que te diga, y comprobarlo.
- Cambiar el color de los badges, por ejemplo para marcar los múltiplos de 3.
- Explicar por qué el `do…while` se repite con 0 y los otros dos bucles no.
- Encontrar y corregir, con ayuda de la consola, un error que yo introduzca en tu código.

Cómo se puntúa

Puntos	Qué demuestras
5	Haces el cambio sin ayuda y con soltura, explicas con seguridad tu código y lo que aparece en la consola, y respondes bien a lo que te pregunto.
4	Haces el cambio sin ayuda, con alguna duda menor, y explicas bien lo esencial.
3	Haces el cambio con una pista pequeña y explicas correctamente la mayor parte de tu código.
2	Necesitas varias pistas para hacer el cambio o tu explicación tiene lagunas importantes.
1	No consigues hacer el cambio sin mucha ayuda y solo sabes explicar partes sueltas de tu código.
0	No consigues hacer el cambio ni explicar tu propio código, o no te presentas.

Se pueden dar valores intermedios de medio punto.

Nota final = rúbrica (máximo 5) + defensa en clase (máximo 5).

Uso de la inteligencia artificial

Puedes apoyarte en herramientas de IA para orientarte, entender un error o revisar la redacción, pero el código y las explicaciones tienen que ser tuyos. Si las utilizas, indícalo al final del README en un apartado «Uso de IA»: qué herramienta, para qué y qué hiciste después con su respuesta. Declararlo no resta nota.

Advertencia: si se detecta que la tarea se ha generado con IA sin trabajo propio, la calificación será un 0 absoluto.

Son indicios: la falta de evidencias propias, el código con técnicas que no hemos visto y que no sabes explicar, los textos genéricos que no responden a lo que se pide, subir todo el trabajo de golpe en un único commit y no saber explicar lo entregado. Una captura tomada de internet, prestada o generada cuenta como el mismo caso.

La defensa en clase sirve precisamente para comprobarlo: si en ella no eres capaz de explicar nada de lo que has entregado, no solo no sumas esos 5 puntos, sino que se considera un indicio claro de que la tarea no es tuya.

Ejemplo de resultado esperado

